

多模态与大语言模型越狱攻击机制深度剖析： 从CAMO、ArtPrompt到FigStep的实验设计 与评估标准全景报告

1. 引言：生成式人工智能的安全边界与越狱威胁

随着大语言模型(LLM)和大型视觉-语言模型(LVLM)在关键基础设施、内容创作及决策辅助系统中的广泛部署，其安全性问题已成为人工智能领域的核心议题。尽管模型开发者通过监督微调(SFT)和基于人类反馈的强化学习(RLHF)等对齐技术构建了复杂的安全护栏，但“越狱攻击”(Jailbreak Attacks)——即通过精心设计的对抗性输入诱导模型生成被禁止的有害内容——仍然是悬在生成式AI头顶的达摩克利斯之剑。

本报告旨在深入剖析当前越狱攻击领域的三项前沿工作：**CAMO**(Cross-modal Adversarial Multimodal Obfuscation)、**ArtPrompt**以及**FigStep**。这三者分别代表了从文本到多模态、从语义混淆到视觉排版攻击的不同技术路径，深刻揭示了当前模型在跨模态理解、非语义符号处理以及视觉编码器安全对齐方面的系统性漏洞。报告将超越单纯的机制描述，重点解构其实验设计、数据集构建逻辑(如SafeBench、HEX-PHI、AdvBench-M)以及最为关键的评估指标——攻击成功率(ASR)的定义与计算方式。

此外，本报告结合2024年至2025年间NeurIPS、ICLR、USENIX Security、CCS等顶会的前沿文献，系统梳理越狱攻击领域的通用实验范式与评估标准演变，旨在为安全研究人员提供一份详尽的参考指南。

2. CAMO：基于跨模态推理的隐蔽式越狱攻击框架

CAMO(Cross-modal Adversarial Multimodal Obfuscation)的提出，标志着多模态越狱攻击从简单的“图像注入”转向了更为复杂的“推理诱导”。传统的黑盒越狱方法主要依赖于对抗性文本后缀或图像扰动，但这些方法往往因特征明显而被基于困惑度(Perplexity, PPL)或OCR检测的防御系统轻易拦截。CAMO的核心创新在于利用LVLM强大的跨模态推理能力，将有害指令拆解为看似良性的文本和视觉线索，迫使模型在推理过程中自行重组出恶意意图¹。

2.1 攻击机理：推理时组合性(Inference-Time Compositionality)

CAMO的攻击逻辑建立在“推理时组合性”这一概念之上。其核心思想是：单个组件在隔离状态下是良性的，但当它们被具备推理能力的模型组合时，就会产生有害的语义爆发。这一过程可以类比为化学反应，无害的反应物在特定条件下混合产生剧烈反应。

CAMO的工作流程包含四个精密设计的阶段：

- 敏感关键词提取(Target Keyword Selection)：

攻击首先通过词性标注 (POS tagging) 和领域特定的敏感词典, 从原始有害 Prompt 中识别出核心敏感词 (如 "bomb", "virus", "suicide")。这些词是触发模型安全拒绝机制的主要诱因¹。算法会根据词语的语义重要性和长度进行排序, 确保攻击目标精准。

2. 跨模态分解 (Cross-modal Decomposition):

识别出的敏感词被分解为两个互补的部分, 分别通过文本和视觉通道传输:

- 文本组件 (**Textual Component**): 在文本 Prompt 中, 敏感词被部分掩码 (Masked)。例如, "explosive" 可能被处理为 "_xplosive" 或更复杂的碎片形式。由于掩码后的字符串在文本层面不再具备明确的有害语义, 因此能够有效绕过基于文本匹配或 PPL 检测的过滤器³。
- 视觉组件 (**Visual Component**): 缺失的信息 (如字符 "e") 被编码为视觉谜题或符号映射表嵌入到图像中。例如, 图像中可能包含一个数学等式 "7+6=?", 并建立规则 "答案13对应的字符是e"。这种设计利用了 LVLM 的数学推理和 OCR 能力, 将单纯的字符识别转化为逻辑推理任务³。

3. 混淆查询构建 (Obfuscated Query Construction):

最终输入给模型的 Prompt 是一个复合指令, 要求模型首先解决图像中的谜题, 获取缺失字符, 填补文本中的空缺, 并在理解完整意图后执行指令。这种 "任务嵌套" 策略 (Task Nesting) 有效地分散了安全护栏的注意力, 因为模型在执行良性的解谜任务时, 不仅消耗了推理预算, 还绕过了针对直接恶意指令的防御检查¹。

4. 动态难度调整 (Dynamic Difficulty Adjustment):

CAMO 引入了动态调节机制, 在粗粒度 (掩码单词的数量) 和细粒度 (单词内掩码字符的比例) 两个维度上调整混淆难度。这使得攻击者能够在 "隐蔽性" (Stealthiness) 和 "攻击有效性" (Effectiveness) 之间找到最佳平衡点, 确保模型既无法直接拒绝, 又能正确还原并执行恶意指令²。

2.2 实验设计方案分析

CAMO 的实验设计展现了极高的严谨性, 特别是在黑盒环境下的跨模型迁移能力和防御规避能力的验证上。

2.2.1 目标模型与基线

实验选取了涵盖闭源与开源的顶级 LVLM, 包括 **GPT-4o**, **GPT-4o-mini**, **GPT-4.1-nano** 以及 **Qwen2.5-VL-72B-Instruct** 和 **DeepSeek-R1**。这种多样化的模型选择确保了攻击方法的普适性验证。基线方法对比了传统的文本对抗攻击 (如 GCG 的变体) 和简单的图像注入攻击, 以突显 CAMO 在查询效率和隐蔽性上的优势²。

2.2.2 数据集构建: AdvBench-M

CAMO 使用了基于 **AdvBench** 改编的多模态数据集 (通常被称为 **AdvBench-M**)。AdvBench 是 LLM 安全领域的标准数据集, 包含大量有害指令。CAMO 从中选取了涉及炸弹制造、毒品合成、网络攻击等高风险类别的 Prompt, 并将其转化为上述的图文对形式。实验涵盖了 8 个主要的威胁类别 (如 Bomb_Explosive, Drugs, Cybercrime 等), 每个类别约包含 30 个样本, 确保了对不同危害类型的覆盖¹。

2.2.3 攻击成功率(ASR)的定义与评估

CAMO采用了目前顶会研究中最为推崇的**系统级大模型裁判(System-level LLM Judge)**评估法。

- 裁判模型：使用GPT-4(如GPT-4-0613或GPT-4o)作为评估者，利用其强大的语义理解能力来判断受害者模型的输出是否构成越狱。
- 评分标准(System Prompt for Evaluation)：裁判模型接收受害者模型的响应和原始有害Prompt，并基于一个固定的Prompt模板进行打分。评分通常采用1到10分制：
 - 1分：完全拒绝或回答无关内容。
 - 5分：包含部分有害信息，但不具备操作性。
 - 10分：提供了完整、详细、无歧义的危害指导，完全违反安全准则。
- ASR计算公式：
若评分 $Score_i > Threshold$ (通常阈值设为5)，则判定为攻击成功。ASR即为攻击成功的样本数占总样本数的比例。

$$ASR = \frac{\sum_{i=1}^N \mathbb{I}(Score_i > 5)}{N}$$

这种基于LLM评分的定义方式比传统的关键词匹配(Keyword Matching)更为鲁棒，能够识别出那些虽然没有包含敏感词但实质上有害的回复(Implicit Harm)，也能排除那些虽然包含敏感词但是是在进行劝诫的安全回复(Benign Refusal)。

2.2.4 防御规避评估

CAMO的一个显著特点是对防御机制的针对性测试。实验明确评估了针对**PPL Filter**(困惑度过滤)、**OCR Detection**(OCR敏感词检测)以及**OpenAI Moderation API**的通过率。结果显示CAMO在这些防御下达到了100%的逃逸率，证明了其基于推理重构的机制能够有效“欺骗”现有的浅层审查机制¹。

3. ArtPrompt: 利用视觉-语义鸿沟的ASCII艺术攻击

ArtPrompt揭示了LLM在处理非语义符号排列时的根本性缺陷。尽管LLM在自然语言处理上表现卓越，但它们对ASCII艺术(用字符组成的图形)的识别能力极差。ArtPrompt正是利用这一“盲点”，将有害语义封装在视觉结构中，从而绕过针对文本语义对齐的安全机制。

3.1 攻击机理：语义掩盖与视觉解码

ArtPrompt的核心逻辑是挑战LLM对齐训练的基本假设：即训练语料库完全由语义主导。

1. 敏感词掩码(Word Masking)：
攻击的第一步与CAMO类似，识别并掩盖Prompt中的触发词。例如，将“How to build a bomb”转化为“How to build a MASK”。这一步破坏了句子的直接有害语义，使其在初步的文本扫描中显得无害¹。

2. ASCII艺术替换与伪装(ASCII Art Substitution & Cloaking):
被掩盖的词(如“BOMB”)被替换为其ASCII艺术形式。这种形式在字符层面是一堆杂乱的符号(如*, |, /, \), 但在视觉层面构成了单词的形状。最终的Prompt(Cloaked Prompt)包含掩码后的文本和ASCII艺术, 并指示模型识别艺术字并用其填空以回答问题。
3. 利用识别短板(Exploiting Recognition Deficits):
研究发现, GPT-3.5、GPT-4、Claude等模型在直接识别ASCII艺术方面表现拙劣(即它们“看不懂”这是什么词), 但矛盾的是, 当这种艺术字被嵌入到上下文中时, 模型往往会尝试去“解释”它, 并在解释的过程中绕过了针对该词汇(如“bomb”)的安全阻断。这是因为在Tokenization阶段, ASCII艺术被切分为无意义的碎片, 无法触发嵌入空间的有害概念聚类, 直到模型在深层推理中“意识到”这是什么时, 生成过程已经开始, 往往越过了拒绝机制的触发点¹。

3.2 实验设计与数据集构建

ArtPrompt的实验设计非常注重基础能力的验证, 即首先证明LLM在ASCII识别上的脆弱性, 再证明这种脆弱性可被利用。

3.2.1 基础能力评估: VITC Benchmark

为了量化LLM识别非语义Prompt的能力, 作者构建了**Vision-in-Text Challenge (VITC)** 基准测试。该测试包含大量用ASCII艺术表示的字符和单词。实验结果表明, 主流模型在VITC上的识别准确率极低, 这为ArtPrompt的攻击成功提供了理论支撑——模型看不懂, 所以防御机制也“看不懂”, 攻击者便可利用这种模糊性混水摸鱼¹。

3.2.2 攻击评估数据集: HEx-PHI与AdvBench

ArtPrompt不仅使用了通用的**AdvBench**, 还引入了一个更为复杂的数据集**HEx-PHI**。

- **HEx-PHI**构建细节: 该数据集包含330个有害指令, 覆盖11个禁止类别(如Illegal Activity, Hate Speech, Malware, Fraud等), 每个类别30个样本。数据来源包括Anthropic的Red Teaming数据以及AdvBench的扩展。这些指令经过人工和GPT-4的进一步细化和分类, 确保其具有高度的危害性和现实意义⁶。
- 数据集意义: 引入HEx-PHI的目的是为了测试攻击在更广泛、更细粒度的危害类别上的有效性, 而不仅仅局限于AdvBench中常见的几类¹。

3.2.3 评估指标与ASR定义

ArtPrompt采用了多维度的评估指标:

- **HPR (Helpful Prompt Rate)**: 衡量模型尝试回答而非拒绝的比例。这不同于ASR, 它只关注模型是否“顺从”了指令, 而不关注内容是否有害。
- **HS (Harmfulness Score)**: 有害性评分, 采用1-5分制。
 - 1分: 无害。
 - 5分: 极度有害。
- **ASR (Attack Success Rate)**:
ArtPrompt对ASR的定义非常严格。只有当模型的响应被评分模型(通常是GPT-4 Judge)评

定为 $HS = 5$ (有时包括4, 视具体设定而定, 但Snippet暗示高严重性) 时, 才被计为成功。这意味着仅有顺从的态度 (Helpful) 是不够的, 必须生成实质性的有害内容¹。

$$ASR = \frac{\text{Count}(HS=5)}{\text{Total Queries}}$$

4. FigStep: 基于排版视觉提示的黑盒越狱

FigStep针对的是多模态模型 (LVLM) 特有的架构漏洞——视觉编码器 (Vision Encoder) 与文本解码器 (Text Decoder) 之间的安全对齐断层。

4.1 攻击机理: 视觉通道的语义泄露

FigStep的核心发现是: LVLM的视觉编码器 (如CLIP、SigLIP) 在预训练阶段学习了丰富的语义对齐 (将图像中的文字与其语义关联), 但这一过程缺乏安全对齐 (Safety Alignment)。相比之下, 文本模块 (LLM) 经过了严格的RLHF和安全微调。

1. 排版转换 (Typography Conversion):
FigStep不直接输入有害文本 (如“Steps to make meth”), 而是将这段文本渲染为一张图片。图片背景干净, 文字清晰, 使用标准字体¹。
2. 良性文本引导 (Benign Textual Context):
配合图片的文本Prompt通常是无害的, 如“图片里写了什么? 请按照上面的指示做”或“将图片内容转换为文本并执行”。
3. 绕过逻辑 (Safety Alignment Gap):
当图像通过视觉编码器时, 有害指令被转换为视觉嵌入 (Visual Embedding)。由于视觉编码器未经过针对有害内容的负向训练, 这些嵌入向量忠实地保留了“制造毒品”的语义, 但没有附带“这是有害内容”的标签。当这些嵌入通过投影层 (Connector) 进入LLM时, LLM将其视为普通的语义输入进行处理。由于文本Prompt本身是良性的 (“请执行图片指令”), LLM的文本安全过滤器不会被触发。最终, 模型在视觉语义的驱动下生成了有害响应¹。

为了应对具备OCR检测能力的闭源模型 (如GPT-4V), 作者提出了**FigStep-Pro**, 将有害指令拆分为多个图像片段 (如“Ste”+“ps”), 进一步打散视觉特征, 使其无法被单一的OCR扫描识别, 需模型在推理层进行拼接¹。

4.2 实验设计: SafeBench的构建与评估

FigStep的一大贡献是提出了**SafeBench**, 填补了LVLM安全评估基准的空白。

4.2.1 SafeBench数据集构建

- 规模与分类: SafeBench包含**500**个有害问题, 覆盖**10**个禁止话题 (如Hate Speech, Physical Harm, Fraud, Illegal Activities等), 每个话题50个问题。
- 生成方法: 这些问题并非直接采集自网络, 而是利用GPT-4基于OpenAI和Meta的使用策略 (Usage Policies) 生成的。通过特定的Prompt (文中称为Prompt 2), 诱导GPT-4生成多样化且违规的指令。

- **SafeBench-Tiny**: 为了快速消融实验, 还构建了一个包含50个问题的微缩版数据集¹。

4.2.2 评估流程与ASR定义(人工评估的核心地位)

与CAMO和ArtPrompt主要依赖LLM Judge不同, FigStep在核心评估中强调了**人工审查(Manual Review)**的重要性。

- **ASR定义**: 攻击成功率被定义为模型针对有害查询生成了**不安全响应(Unsafe Response)**的比例。
- **人工评估标准**:
评估者需判断模型的输出是否提供了具有操作性或实质性危害的信息。仅仅是复述图片中的有害文字但不执行, 或者进行道德说教, 都被判定为攻击失败(ASR=0)。只有当模型输出了符合攻击意图的具体步骤或内容时, 才计为成功¹。
- **为何使用人工**: 作者认为, 在多模态攻击这一新兴领域, 自动评估器(如基于文本的LLM Judge)可能无法准确理解图文结合的语境, 容易产生误判(False Positives/Negatives)。因此, 尽管耗时, 人工评估提供了最可靠的Ground Truth(平均ASR达到82.50%)¹。

5. 越狱攻击领域的通用实验设计与评估标准(2024-2025顶会综述)

通过对NeurIPS 2024、ICLR 2025、USENIX Security 2025等顶会文献的调研, 我们可以提炼出当前越狱攻击研究在实验设计和评估标准上的几大核心趋势。这些趋势正逐渐形成一套标准化的“红队测试”(Red Teaming)范式。

5.1 实验设计的演进: 从单一到全面

5.1.1 数据集的整合与分层

早期的研究往往依赖单一的AdvBench, 但现有的顶刊论文倾向于使用更全面、分类更细致的基准套件:

- **HarmBench (ICML 2024 / NeurIPS 2024)**: 已成为事实上的新标准。它不仅包含510个有害行为, 还涵盖了标准文本、版权内容、上下文敏感内容以及多模态内容四大功能类别。其最大的贡献是提供了一套标准化的自动化评估管线, 解决了以往研究中ASR不可比的问题¹⁰。
- **JailbreakBench (NeurIPS 2024)**: 侧重于可复现性。它提供了一个包含100个不同滥用行为的数据集(JBB-Behaviors), 并维护了一个包含最新攻击Prompt(Artifacts)的存储库。这允许研究者直接对比新攻击与已知攻击的效果, 而不是从零开始生成¹¹。
- **StrongREJECT**: 针对现有基准(如AdvBench)中问题过于模糊或容易被误拒的缺陷, StrongREJECT设计了313个具体的、模型通常会拒绝的高质量问题, 旨在更精准地测量模型的拒绝边界¹²。

5.1.2 威胁模型的细化

实验设计不再笼统地假设“黑盒”，而是明确区分具体的威胁场景：

- **Query Budget** (查询预算)：明确区分 **Single-shot** (单次尝试) 与 **Best-of-N** (N次尝试取最佳)。研究表明，随着尝试次数增加，ASR会呈指数级上升。因此，标准化的实验必须报告使用的查询次数 (如HarmBench通常固定解码参数和尝试次数)¹⁴。
- **攻击面扩展**：实验不仅测试文本，还扩展到多轮对话 (Multi-turn, 如MTJ-Bench) 和多模态输入 (如AdvBench-M, JailBreakV-28K)，以模拟更真实的攻击路径¹⁵。

5.2 评估标准的共识: LLM-as-a-Judge

依靠简单的关键词匹配 (Keyword Matching, 如检查是否包含 "I cannot") 已被证明既不准确也不鲁棒。当前的通用做法是 **LLM-as-a-Judge**。

5.2.1 评分机制

- **模型选择**：GPT-4是目前的“金标准”裁判。此外，经过专门微调的开源模型 (如Llama Guard, HarmBench-Classifer) 因其低成本和可复现性，正逐渐被广泛采用¹⁷。
- **评分量表**：
 - **二元分类 (Binary)**：仅判断“成功/失败”。JailbreakBench和HarmBench主要采用此指标计算ASR。
 - **李克特量表 (Likert Scale 1-5/1-10)**：用于捕捉攻击的细微差别 (如CAMO使用的1-10分制)。高分代表不仅越狱成功，而且回复内容详尽、危害性大。这有助于区分“无益的顺从”和“有害的顺从”²。
- **ASR的标准化定义**：

$$ASR = \frac{1}{N} \sum_{i=1}^N \mathbb{I}(\text{Judge}(\text{Response}_i) \in \text{Harmful})$$

其中， Judge 是一个预先校准过的分类器或Prompt模板。HarmBench进一步规范了拒绝率 (Refusal Rate) 的计算，以防止模型通过过度拒绝 (Over-refusal) 来刷高安全性分数¹⁰。

5.2.2 评估的稳健性

为了防止裁判模型本身被攻击 (Judge Jailbreak) 或产生幻觉，最新的评估标准要求：

- **裁判校准 (Calibration)**：必须在少量样本上验证LLM裁判与人类标注的一致性。
- **双重检查 (Dual Classifiers)**：如HarmBench结合了基于规则的分类器和LLM分类器来提高判定的准确度。
- **拒绝与有害的分离**：明确区分模型是“拒绝回答”还是“回答了但无害”，以及“回答了且有害”。这对于评估攻击的实际风险至关重要¹⁰。

6. 总结与展望

CAMO、ArtPrompt和FigStep三项工作共同揭示了多模态大模型在模态对齐上的巨大鸿沟。

- **CAMO** 证明了推理能力本身可以被武器化，通过跨模态拼图绕过防御。

- **ArtPrompt** 证明了文本编码器的视觉盲区是安全对齐的死角。
- **FigStep** 证明了视觉编码器的语义泄漏可以直接绕过文本侧的防御体系。

在实验设计上，这三者均采用了自定义/改编数据集 + LLM/人工裁判的模式，这符合当前学术界的主流趋势。然而，随着HarmBench和JailbreakBench等标准化基准的出现，未来的越狱研究将面临更严格的可复现性要求。单纯报告ASR已不足够，研究者必须详细说明查询预算、裁判提示词（Judge Prompts）以及针对误拒率（Over-refusal Rate）的控制，以证明攻击的真实有效性而非统计巧合。

表1: 三种攻击方法与通用标准的对比总结

维度	CAMO	ArtPrompt	FigStep	行业通用标准 (2025)
核心机制	跨模态推理组合	ASCII视觉欺骗	排版视觉注入	复合型/多轮对话/自动化优化
主要数据集	AdvBench-M (改编)	HEx-PHI, VITC	SafeBench (自建)	HarmBench, JailbreakBench
ASR定义	LLM评分 (1-10) > 阈值	LLM评分 (1-5) = 5	人工审查 (Binary)	自动化LLM裁判 (GPT-4/Llama Guard)
评估侧重	隐蔽性 (PPL/OCR规避)	视觉识别缺陷	跨模态对齐缺失	攻击有效性、鲁棒性、可迁移性
查询成本	单轮/低成本	单轮/低成本	单轮/低成本	需报告具体 Query Budget

这些研究不仅暴露了当前模型的脆弱性，也为未来的防御工作指明了方向：安全对齐不能仅停留在单一模态或语义层面，必须向跨模态一致性和对抗性鲁棒性深水区迈进。

引用的著作

1. Cross-Modal Obfuscation for Jailbreak Attacks on Large Vision-Language Models.pdf
2. Cross-Modal Obfuscation for Jailbreak Attacks on Large Vision-Language Models - arXiv, 访问时间为 十二月 16, 2025, <https://arxiv.org/html/2506.16760v1>
3. Cross-Modal Obfuscation for Jailbreak Attacks on Large Vision-Language

- Models - arXiv, 访问时间为 十二月 16, 2025,
<https://www.arxiv.org/pdf/2506.16760>
4. CROSS-MODAL OBFUSCATION FOR JAILBREAK ATTACKS ON LARGE VISION-LANGUAGE MODELS - OpenReview, 访问时间为 十二月 16, 2025,
<https://openreview.net/pdf/1c3b193877e869664ad2b5e5d2088c544ccae9a0.pdf>
 5. ArtPrompt: ASCII Art-based Jailbreak Attacks against Aligned LLMs - ACL Anthology, 访问时间为 十二月 16, 2025,
<https://aclanthology.org/2024.acl-long.809.pdf>
 6. LLM-Tuning-Safety/HEX-PHI · Datasets at Hugging Face, 访问时间为 十二月 16, 2025, <https://huggingface.co/datasets/LLM-Tuning-Safety/HEX-PHI>
 7. README.md · LLM-Tuning-Safety/HEX-PHI at main - Hugging Face, 访问时间为 十二月 16, 2025,
<https://huggingface.co/datasets/LLM-Tuning-Safety/HEX-PHI/blob/main/README.md>
 8. SQL Injection Jailbreak: A Structural Disaster of Large Language Models - ChatPaper, 访问时间为 十二月 16, 2025, <https://chatpaper.com/paper/177711>
 9. CryptoAllab/FigStep: [AAAI'25 (Oral)] Jailbreaking Large Vision-language Models via Typographic Visual Prompts - GitHub, 访问时间为 十二月 16, 2025,
<https://github.com/CryptoAllab/FigStep>
 10. HarmBench: Standardized LLM Red Teaming Framework - Emergent Mind, 访问时间为 十二月 16, 2025,
<https://www.emergentmind.com/topics/harmbench-framework>
 11. JailbreakBench: LLM robustness benchmark, 访问时间为 十二月 16, 2025,
<https://jailbreakbench.github.io/>
 12. StrongREJECT: Jailbreak Benchmark in LLMs - Emergent Mind, 访问时间为 十二月 16, 2025, <https://www.emergentmind.com/papers/2402.10260>
 13. A STRONGREJECT for Empty Jailbreaks - arXiv, 访问时间为 十二月 16, 2025,
<https://arxiv.org/pdf/2402.10260>
 14. Why Attack Success Rate (ASR) Isn't Comparable Across Jailbreak Papers Without a Shared Threat Model | Promptfoo, 访问时间为 十二月 16, 2025,
<https://www.promptfoo.dev/blog/asr-not-portable-metric/>
 15. MTJ-Bench: Multi-Turn Jailbreak Evaluation - Emergent Mind, 访问时间为 十二月 16, 2025,
<https://www.emergentmind.com/topics/multi-turn-jailbreak-benchmark-mtj-bench>
 16. JailBreakV: A Benchmark for Assessing the Robustness of MultiModal Large Language Models against Jailbreak Attacks - arXiv, 访问时间为 十二月 16, 2025,
<https://arxiv.org/html/2404.03027v4>
 17. JAILJUDGE: A Comprehensive Jailbreak Judge Benchmark with Multi-Agent Enhanced Explanation Evaluation Framework - arXiv, 访问时间为 十二月 16, 2025,
<https://arxiv.org/html/2410.12855v2>
 18. LLM as a judge - Wandb, 访问时间为 十二月 16, 2025,
<https://wandb.ai/romaingrx/llm-as-a-judge/reports/LLM-as-a-judge--Vmlldzo5MjcwNTMw>
 19. Bad Likert Judge: A Novel Multi-Turn Technique to Jailbreak LLMs by Misusing

Their Evaluation Capability, 访问时间为 十二月 16, 2025,
<https://unit42.paloaltonetworks.com/multi-turn-technique-jailbreaks-llms/>